WEB3 SMART CONTRACT SECURITY REVIEW

UNDER-CONSTRAINED CIRCOM CIRCUIT

Circom · Zero-knowledge · snarkjs

Manual review + working proof-of-concept + severity-rated report

High 1

GILLES MUSY

Smart contract security researcher

Confirmed findings · Code4rena GiMu84 · Cantina GilMu

1 Disclaimer

This document is a demonstration security review performed on Square.circom, a deliberately vulnerable sample authored by Gilles Musy for portfolio purposes. It is not a client engagement: the code was never deployed, holds no user funds, and the vulnerabilities it contains were intentionally introduced to illustrate review methodology, severity classification and proof-of-concept construction. The findings, proofs of concept and remediation guidance below are genuine analysis of the code as written.

A security review is a time-, resource- and expertise-bound effort to surface as many issues as possible; it cannot prove the absence of vulnerabilities. Subsequent reviews, a bug-bounty program and runtime monitoring are recommended for any production deployment.

2 Risk classification

Severity is derived from the impact of an issue and the likelihood of its occurrence.

IMPACT ▼ / LIKELIHOOD ▶	HIGH	MEDIUM	LOW
HIGH	Critical	High	Medium
MEDIUM	High	Medium	Low
LOW	Medium	Low	Low

IMPACT High: significant loss of assets or harm to many users. Medium: moderate loss or harm. Low: minor loss or harm.

LIKELIHOOD High: practical under realistic on-chain conditions at low cost. Medium: conditionally incentivised but plausible. Low: needs unlikely assumptions or significant attacker stake.

ACTION REQUIRED Critical: must fix as soon as possible. High: must fix. Medium: should fix. Low: could fix.

INFORMATIONAL & GAS Code-quality observations and gas optimisations, reported separately in section 5. These are not vulnerabilities and are not scored on the matrix above.

3 About the target

`Square.circom`, a Circom circuit proving knowledge of a square root, with a Groth16 backend.

TYPE Demonstration review on intentionally vulnerable code

METHOD snarkjs proof-of-concept, vulnerable branch (`master`) and remediated branch (`fixed`)

4 Findings

HIGH H-01, Under-constrained signal breaks soundness (High)

The circuit is meant to prove knowledge of a private `root` such that `root^2 == n`, where `n` is public. It does not. A signal is assigned without being constrained, so a prover can produce a verifying proof using a `root` that is not a square root of `n`. In the proof-of-concept the public input is `n = 9` and the forged proof uses `root = 7`, even though `7^2 = 49`. The verifier accepts it.

ROOT CAUSE

Circom distinguishes assignment from constraint:

- `<--` assigns a value to a signal in the witness. It adds nothing to the R1CS.
- `<==` assigns and adds the corresponding constraint.
- `===` adds a constraint without assigning.

The circuit uses `<--` where it needed `<==`:

```
template Square() {
    signal input root;   // private
    signal input n;      // public
    signal sq;
    sq <-- root * root;  // assigns sq, but does NOT constrain sq == root*root
    sq === n;
}
```

The only constraint emitted is `sq === n`. The relation `sq == root * root` is never added, so `root` appears in no constraint at all. Compiled with `--O0`, the R1CS has a single constraint and `root` is free; under the default optimizer the circuit reduces to zero constraints, which is the same defect seen at its limit. Either way, the prover is free to choose `root`.

ATTACK

The witness vector is `[1, n, root, sq]`. An honest prover for `n = 9` supplies `root = 3`, the witness calculator sets `sq = 9`, and `sq === n` holds. To forge:

1. Generate the honest witness, then overwrite only the `root` slot with `7`. Leave `sq = 9`.
2. The forged witness `[1, 9, 7, 9]` still satisfies the sole constraint `sq === n` (9 == 9).
3. Run `groth16.prove` on the forged witness. snarkjs trusts the witness and produces a proof.
4. The verifier checks the proof against the public input `n = 9` and accepts it.

The proof now attests "I know a square root of 9" while the prover used 7. Used as an authorization gate, this is a full bypass.

PROOF OF CONCEPT

`test/exploit.cjs`, run on `master`. It builds the forged witness by serializing a `.wtns` file directly, then proves and verifies:

```
public n              : 9
forged root (7^2=49)  : 7
honest proof verifies : true
forged proof verifies : true
```

The honest proof verifies (the circuit works for a real prover), and the forged proof also verifies (soundness is broken).

RECOMMENDATION

Bind `sq` with `⟸` so the constraint `sq === root * root` is emitted:

```
sq ⟸ root * root;
sq === n;
```

The R1CS then enforces `root * root === n`. On the `fixed` branch the same proof-of-concept reports `forged proof verifies : false`: the forged witness violates `sq === root * root` (9 != 49), so its proof fails verification while the honest proof still passes. As a general rule, every signal a circuit's guarantee depends on must be pinned by a constraint (`⟸` or an explicit `===`). A `⟵` assignment with no accompanying constraint is the canonical under-constrained-circuit bug and should be flagged on sight.

SEVERITY

High. A soundness break lets a prover produce a verifying proof for a statement they cannot satisfy. In a circuit acting as an authorization or state-transition gate, the property the proof was meant to guarantee is fully bypassed.

---

5 Informational & Gas

INFORMATIONAL I-01, Inputs lack range constraints (Informational)

`root` and `n` are field elements with no range or bit-length constraints (e.g. `Num2Bits`). The circuit silently accepts any field-overflowing assignment, and the expected value domain is neither documented nor enforced. If callers assume bounded integers, constrain the inputs explicitly.

INFORMATIONAL I-02, No integration guidance (Informational)

The template ships without notes on witness generation or how `n` is meant to be derived client-side. For a circuit whose security depends on correct usage, document the intended proving flow so integrators do not reintroduce the soundness gap from the consumer side.

> No gas optimisations apply to a Circom circuit; the equivalent axis is constraint count, already minimal here.

Scope and disclaimer

`Square.circom` is intentionally vulnerable code written to demonstrate audit methodology end to end. It is not production code and must never be deployed. The finding above is a real vulnerability in this demo circuit, reproduced with an executable proof-of-concept, not an invented severity.